```
import smtplib
import gpgme
from email import encoders
from email.mime.base import MIMEBase
from email.mime.multipart import MIMEMultipart
from StringIO import StringIO
```

StringIO is a file-like class for manipulating a string buffer. It is, essentially, a memory file.

You will need to create a multi-part Mime formatted message with the attachment you wish to e-mail (see *docs.python.org/library/email-examples.html* for more details). Assume that you are attaching a PDF file:

```
def create_message(filename):
    outer = MIMEMultipart()
    fp = open(filename, 'rb')
    msg = MIMEBase('application', 'pdf')
    msg.set_payload(fp.read())
    fp.close()
    encoders.encode_base64(msg)
    msg.add_header('Content-Disposition', 'attachment', filename=filename)
    outer.attach(msg)
    return outer.as_string()
```

You will next encrypt the message:

```
def encrypt_payload(in_msg, out_msg):
    ctx = gpgme.Context()
    ctx.armor = True
    recipient_key = ctx.get_key('friend@example.com')
    ctx.encrypt([recipient_key], gpgme.ENCRYPT_ALWAYS_TRUST, in_msg,
out_msg)
```

Now, you will need to create another multi-part Mime message that has the encrypted content as the payload. The Mime body must consist of exactly two parts, the first with the content type "application/pgp-encrypted". This part contains the control information. The second part contains the encrypted content as an octet-stream.

```
def mime_pgp_message(fp):
    outer = MIMEMultipart(_subtype='encrypted',protocol='application/
pgp-encrypted')
    outer['Subject'] = 'Attached Encrypted - 5'
    outer['To'] = 'friend@example.com'
    outer['From'] = 'user@example.com'
    msg = MIMEBase('application', 'pgp-encrypted')
    outer.attach(msg)
    enc_part = MIMEBase('application','octet-stream', name='encrypted.
asc')
    fp.seek(0)
    enc_part.set_payload(fp.read())
    outer.attach(enc_part)
    return outer.as_string()
```

Now, you are ready to send the message:

```
def send_message(sender, recipients, composed):
    s = smtplib.SMTP()
    s.connect()
    s.sendmail(sender, recipients, composed)
    s.close()
You would be calling the above routines as follows:
in_msg = StringIO(create_message('Open.pdf'))
out_msg = StringIO()
encrypt_payload(in_msg, out_msg)
composed = mime_pgp_message(out_msg)
send_message('user@example.com', ['friend@example.com'], composed)
```

Unfortunately, signing the document introduces one more level of complexity. Before encrypting the message, you would need to sign it. For this, too, a multi-part message with two parts in the body, is required. So, the steps would be:

1. Create the Mime message
2. PGP Sign the Mime message
3. Create a multi-part Mime message with protocol application/pgp-signature
4. PGP encrypt the signed Mime message
5. Create a multi-part Mime message with protocol application/pgp-encrypted
6. Send the message

Final words

This article turned out to be much harder to write than I expected, as I could not find any tutorials or simple documentation on using *gpgme* or Mime/PGP-encrypted, whether for Python or any other language. In case anyone knows of any, I would love to hear about it.

It is a pity that banks force us to change our passwords every few months. We also need to ensure that our passwords are not the same at all sites. Nor the same as the ones used on the previous few occasions. In short, keeping track of passwords is one horrendous problem. Desktop tools that help use the appropriate password for each application or website are a solution for this problem.

A public key environment would, unambiguously, shift the task of preventing any leakage of passwords from the host sites to the user only. The critical advantage is that we need to protect only one private password. Last but not least, it will save us from making hundreds of enemies because we used our Gmail password at a social networking site, involuntarily inflicting our friends with the "I want to be your friend" spam. END

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com